



Research Intake 2026

Day 7

Introduction to Neural Networks

A Beginner's Guide for Students

Understanding AI, Machine Learning, and Deep Learning

Gudsky Research Foundation

Table of Contents

1. What Is a Neural Network?
2. Activation Functions
3. The Forward Pass & Loss Functions
4. Backpropagation & Gradient Descent
5. The Complete Training Loop
6. Hands-On Lab — Build in NumPy



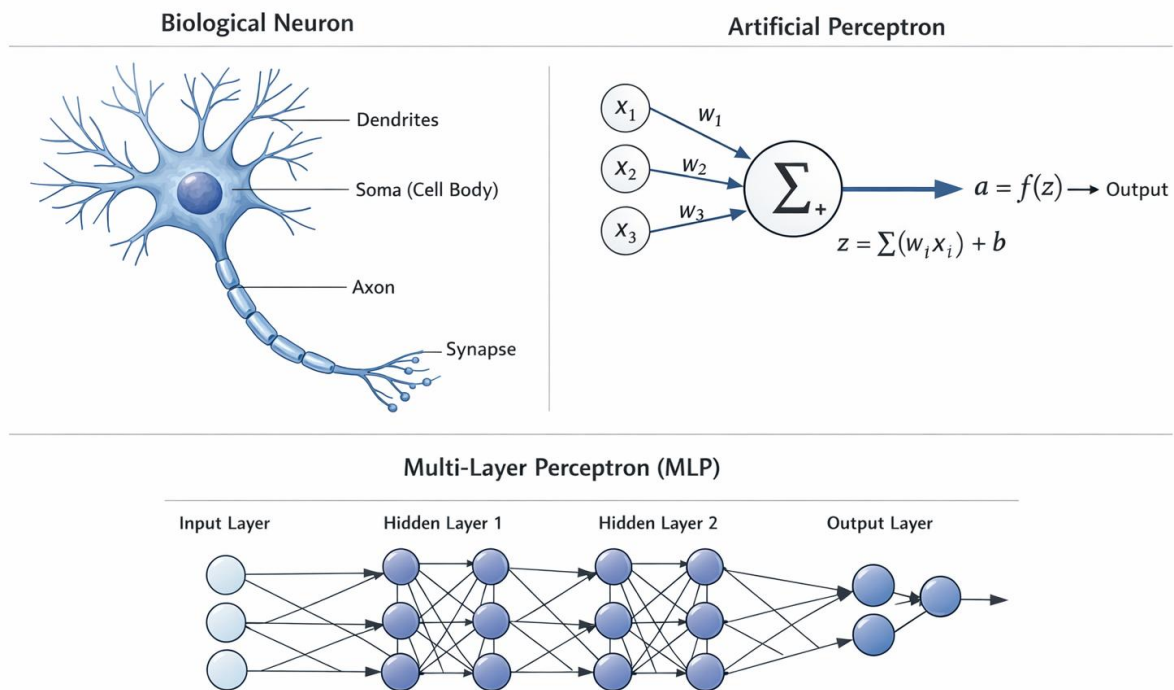
Learning Objectives

By the end of this session, students will be able to:

1. Explain the biological inspiration behind ANNs and map each neuron component to its mathematical equivalent.
2. Define and compute the output of a single perceptron for a given input, weight vector, and bias.
3. Distinguish between linear and non-linear decision boundaries and explain why multi-layer networks overcome single-perceptron limitations.
4. Describe the MLP architecture, correctly naming each layer type and its role.
5. Identify and compare the five core activation functions (Sigmoid, Tanh, ReLU, Leaky ReLU, Softmax) and select the right one for each context.
6. Trace the forward pass step-by-step through a small network, computing activations at each layer.
7. Calculate MSE, Binary Cross-Entropy, and Categorical Cross-Entropy loss and interpret their values.
8. Explain backpropagation conceptually using the chain rule of calculus.
9. Describe how gradient descent updates weights and explain the effect of the learning rate.
10. Implement a simple 2-layer neural network in Python/NumPy from scratch.

1.1 Biological Inspiration & the Neuron Analogy

The human brain contains approximately 86 billion neurons, each connected to thousands of others through junctions called synapses. Understanding how biological neurons process information directly inspired the mathematical design of artificial neural networks (ANNs).



How a biological neuron works:

- Dendrites — branch-like structures that receive incoming electrical signals from other neurons.
- Cell body (soma) — accumulates the incoming signals. If the total input crosses a threshold, the neuron fires.
- Axon — carries the output signal forward to the next neurons via synaptic connections.

Mapping biology to mathematics:

Biological Neuron	Artificial Neuron (Perceptron)
Dendrites	Input values ($x_1, x_2, \dots x_n$)
Synaptic strength	Weights ($w_1, w_2, \dots w_n$)
Cell body summation	Weighted sum: $z = \sum (w_i \cdot x_i) + b$
Firing threshold	Activation function: $a = f(z)$

Axon output	Output value passed to the next layer
-------------	---------------------------------------

💡 Why the Analogy Matters

Training a neural network adjusts weights until firing patterns produce correct outputs — exactly like learning in the brain, which strengthens useful synaptic connections and weakens unused ones.

Weights mirror synaptic strengths: large positive weight = strong excitatory connection; large negative weight = strong inhibitory connection.

1.2 The Perceptron — The Simplest Neural Unit

A perceptron is a single artificial neuron — the fundamental building block of every neural network, regardless of how large. It takes multiple inputs, multiplies each by a learnable weight, sums the results, adds a bias, and passes the total through an activation function.

📖 Formula

Weighted sum: $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$

Output: $a = f(z)$

In matrix form: $z = W \cdot X + b$ then $a = f(z)$

Understanding each component:

- Input vector $X = [x_1, x_2, \dots, x_n]$ — raw features fed into the neuron (e.g., pixel values, sensor readings, word embeddings).
- Weight vector $W = [w_1, w_2, \dots, w_n]$ — learnable parameters. Each weight controls how much influence its input has. A large positive weight amplifies; a large negative weight suppresses.
- Bias b — a single learnable scalar. It shifts the activation threshold, allowing the neuron to fire even when all inputs are zero. Without bias, every decision boundary must pass through the origin — a severe restriction.
- Activation function $f(\cdot)$ — introduces the non-linearity that makes deep networks powerful (covered in Section 02).

🔍 Perceptron for AND Gate

Inputs: $x_1=1, x_2=1$ | Weights: $w_1=0.5, w_2=0.5$ | Bias: $b=-0.7$

Weighted sum: $z = (0.5 \times 1) + (0.5 \times 1) + (-0.7) = 0.5 + 0.5 - 0.7 = 0.3$

Apply step activation: $f(0.3) = 1$ ✓ correct: $\text{AND}(1,1) = 1$

Test with $x_1=1, x_2=0$: $z = 0.5 + 0 - 0.7 = -0.2 \rightarrow f(-0.2) = 0$ ✓ correct: $\text{AND}(1,0) = 0$

📝 The XOR Problem — The Perceptron's Fatal Flaw

A single perceptron can only draw a linear decision boundary (a straight line in 2D, a hyperplane in nD). XOR is not linearly separable: no single line can separate $(0,0) \rightarrow 0$, $(0,1) \rightarrow 1$, $(1,0) \rightarrow 1$, $(1,1) \rightarrow 0$. This limitation, proven by Minsky & Papert (1969), caused the first AI Winter. The solution is to stack layers. Stacking perceptrons creates curved, complex, non-linear boundaries — capable of solving XOR and far harder problems.

1.3 Multi-Layer Perceptron (MLP) — Architecture & Depth

A Multi-Layer Perceptron stacks multiple layers of perceptrons. This architecture creates a network deep enough to learn highly complex, non-linear patterns and is the foundation of modern deep learning.

Layer Type	Position	Role	Neuron Count
Input Layer	First	Receives raw features — no computation performed here	Equal to number of input features
Hidden Layer(s)	Middle	Learns intermediate representations; detects increasingly abstract patterns	Hyperparameter — you decide
Output Layer	Last	Produces the final prediction; activation depends on task type	Depends on the task (see below)

Output layer size by task type:

- Binary Classification → 1 neuron with Sigmoid activation (outputs $P(\text{class}=1)$)
- Multi-class Classification → N neurons with Softmax activation (N = number of classes)
- Regression → 1 or more neurons with no activation (outputs a continuous value)

💡 Why Depth Enables Power

Universal Approximation Theorem: Even a single hidden layer can approximate any continuous function — but may need exponentially many neurons.

With multiple layers, the network builds hierarchical representations: Layer 1 detects edges/curves; Layer 2 detects shapes/textures; Layer 3+ detects objects/concepts.

Depth is dramatically more parameter-efficient than width for learning structured real-world data such as images, text, and audio.

Without an activation function, stacking layers produces nothing more than a single linear transformation — no matter how many layers are added. This is because the composition of linear functions is always linear. Activation functions introduce non-linearity, enabling networks to represent curves, complex boundaries, and real-world patterns.

2.1 Sigmoid



Formula

$$\sigma(z) = 1 / (1 + e^{(-z)}) \quad \text{Output range: } (0, 1)$$

Squashes any real number into a value between 0 and 1. Values far below zero map to ~0; values far above zero map to ~1; $z=0$ maps to 0.5. Historically the first activation used in neural networks.

Strengths:

- Output is interpretable as a probability — natural for binary classification output.
- Smooth and differentiable everywhere — gradients exist at all points.

Weaknesses:

- Vanishing Gradient: For large $|z|$, the gradient approaches zero. In deep stacked Sigmoid networks, these near-zero gradients multiply together during backpropagation, making early layers nearly unable to learn.
- Not zero-centred: Outputs are always positive (0 to 1). Gradients always carry the same sign, which can slow convergence.

When to use:

Binary classification output layer only. Almost never used in hidden layers in modern deep networks.

2.2 Tanh (Hyperbolic Tangent)



Formula

$$\tanh(z) = (e^z - e^{(-z)}) / (e^z + e^{(-z)}) \quad \text{Output range: } (-1, 1)$$

Similar to Sigmoid but zero-centred. Outputs range from -1 to $+1$, with $z=0$ mapping to 0. A significant improvement over Sigmoid for hidden layers.

Strengths:

- Zero-centred — gradients can flow in both positive and negative directions, enabling faster learning.
- Larger gradients than Sigmoid near zero — more efficient updates close to the decision boundary.

Weaknesses:

- Still saturates: For $|z| > 2-3$, the gradient approaches zero. The vanishing gradient problem is reduced but not eliminated.

When to use:

Hidden layers in shallow networks; Recurrent Neural Networks (RNNs) where zero-centred outputs matter.

2.3 ReLU — Rectified Linear Unit

Formula

$\text{ReLU}(z) = \max(0, z)$ Output range: $[0, \infty)$

Extremely simple — outputs the input directly if positive, or zero if negative. Think of it as a switch: off for negatives, on for positives. The dominant activation function in modern deep learning.

Strengths:

- Does NOT saturate for positive inputs — gradient is always exactly 1 for $z > 0$. This largely eliminates the vanishing gradient problem.
- Computationally cheap — just a max operation; no expensive exponential computations.
- Sparse activation — ~50% of neurons output zero on average, creating an efficient, sparse representation.
- Converges much faster in practice than Sigmoid/Tanh — a key reason AlexNet (2012) was a breakthrough.

Weaknesses:

- Dying ReLU Problem: If a neuron's weighted sum is always negative for every training sample, its gradient is permanently zero and it never updates — the neuron is 'dead'. More common with high learning rates.

When to use:

Hidden layers in virtually all deep feedforward and convolutional networks. The default first choice.

2.4 Leaky ReLU & Variants

Formula

Leaky ReLU(z) = $\max(\alpha \cdot z, z)$ α is a small constant (e.g., 0.01)

Parametric ReLU (PReLU): α is a learnable parameter

ELU (Exponential Linear Unit): z for $z \geq 0$ | $\alpha(e^z - 1)$ for $z < 0$

Leaky ReLU fixes the Dying ReLU problem by allowing a small, non-zero gradient ($\alpha \cdot z$) when $z < 0$, so neurons still receive update signals even when inactive.

- Leaky ReLU: simple fix with a fixed small slope for negatives.
- PReLU: adapts α per neuron automatically during training — more flexible.
- ELU: smooth negative region with soft saturation; can improve learning stability.

When to use:

When Dying ReLU is causing training instability. PReLU preferred when extra compute is available.

2.5 Softmax — For Multi-Class Output

Formula

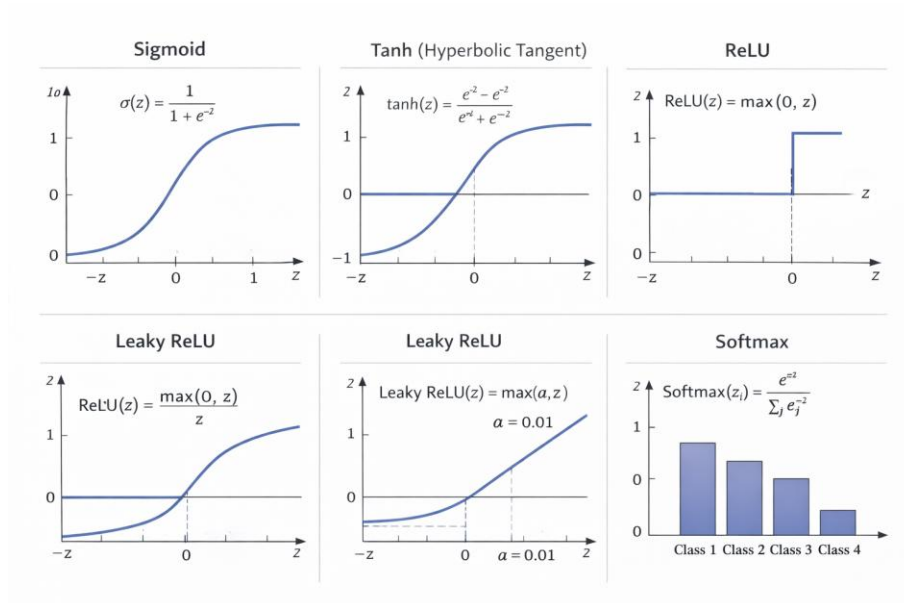
$\text{Softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$ for $i = 1, 2, \dots, K$ classes

Output: A probability distribution across K classes.

All outputs are in $(0, 1)$ and always sum to exactly 1.0.

Applied to the output layer for multi-class classification. Converts raw scores (logits) into a probability distribution — the class with the highest probability is the prediction.

- Mathematically paired with Categorical Cross-Entropy loss — they are designed to work together.
- Not used in hidden layers — only at the output layer of a multi-class classifier.



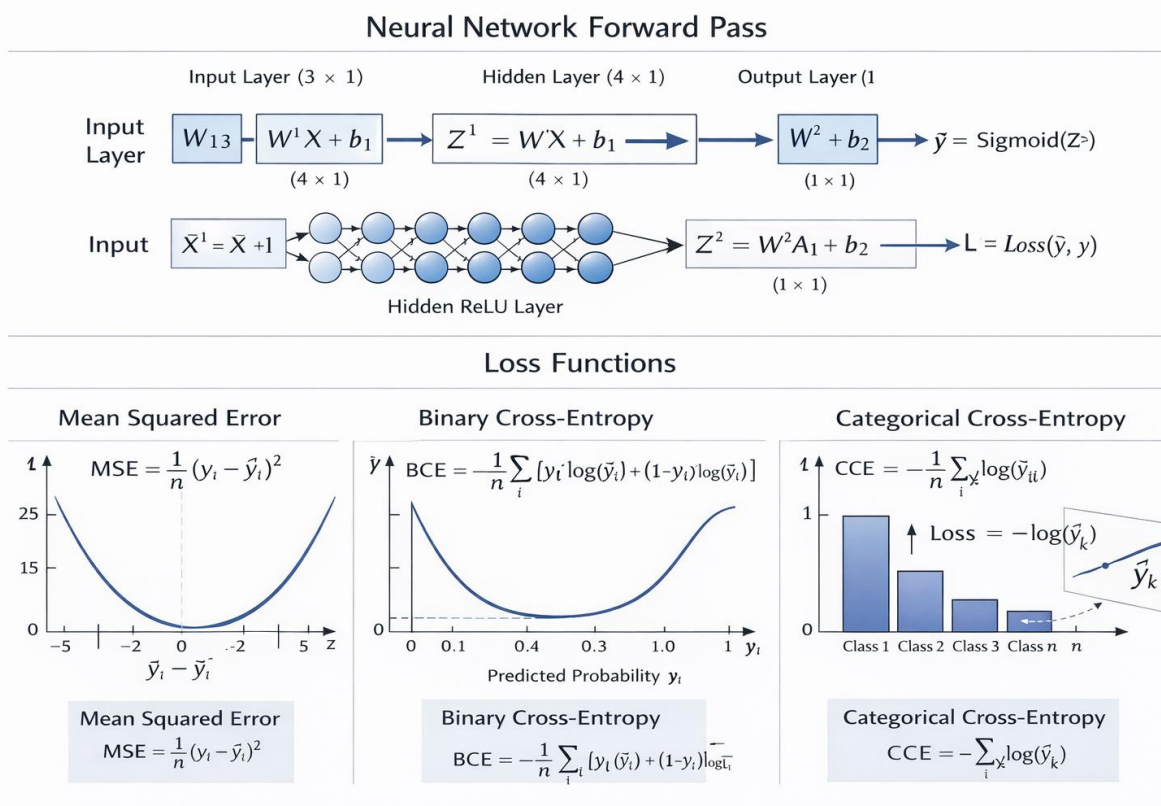
2.6 Activation Functions — Comparison Table

Activation	Formula / Range	Gradient Behaviour	Best Use Case	Key Limitation
Sigmoid	$1/(1+e^{-z}) \rightarrow (0,1)$	Vanishes at extremes	Binary output layer	Vanishing gradient; not zero-centred
Tanh	$(e^z - e^{-z}) / (e^z + e^{-z}) \rightarrow (-1,1)$	Better; still vanishes at extremes	Shallow hidden; RNNs	Still saturates
ReLU	$\max(0, z) \rightarrow [0, \infty)$	1 for $z > 0$; 0 for $z \leq 0$	Hidden layers (default)	Dying neurons
Leaky ReLU	$\max(\alpha z, z) \rightarrow (-\infty, \infty)$	α for $z < 0$; 1 for $z > 0$	When ReLU neurons die	Manual hyperparameter α
Softmax	$e^{z_i} / \sum_j e^{z_j} \rightarrow (0,1) \text{ sum}=1$	Coupled across all outputs	Multi-class output only	Not for hidden layers

03

The Forward Pass & Loss Functions

The forward pass is the process of feeding input data through every layer of the network — from input to output — to produce a prediction. It must be completely understood before studying how the network learns (backpropagation).



3.1 Step-by-Step: Forward Pass Through a 3-Layer MLP

Network setup: Input (3 neurons) → Hidden (4 neurons, ReLU) → Output (1 neuron, Sigmoid)

Forward Pass — Step by Step

Step 1 — Receive input:

$$X = [x_1, x_2, x_3] \quad (\text{shape: } 3 \times 1)$$

Step 2 — Hidden layer linear step:

$$Z^1 = W^1 \cdot X + b^1$$

W^1 is (4×3) , X is (3×1) → Z^1 is (4×1) (4 values, one per hidden neuron)

Step 3 — Hidden layer activation:

$$A^1 = \text{ReLU}(Z^1) \quad (\text{clips all negative values to } 0)$$

Step 4 — Output layer linear step:

$$Z^2 = W^2 \cdot A^1 + b^2$$

W^2 is (1×4) , A^1 is (4×1) → Z^2 is (1×1) (single raw score / logit)

Step 5 — Output layer activation:

$$\hat{y} = \text{Sigmoid}(Z^2) \quad (\text{squashes to } (0,1); P(\text{class}=1))$$

Step 6 — Compute loss:

$$L = \text{Loss}(\hat{y}, y) \quad (\text{measures how wrong the prediction is})$$

💡 Dimensions Always Matter

Every layer's output shape must match the next layer's expected input shape.

If W^1 is (4×3) and X is (3×1) : $Z^1 = W^1 \cdot X \rightarrow$ shape (4×1) . This gives 4 hidden activations. ✓

Matrix dimension mismatch errors are the most common beginner bug — always verify shapes before debugging.

3.2 Loss Functions — Measuring Prediction Error

The loss function (also called cost or objective function) quantifies how far the network's prediction $\hat{y}$ is from the true label y . The entire goal of training is to minimise this value.

Mean Squared Error (MSE) — for Regression

📖 Formula

$$\text{MSE} = (1/n) \cdot \sum_i (y_i - \hat{y}_i)^2$$

n = number of training samples in the batch

Computes the average of squared differences between true and predicted values. Squaring achieves two things: it makes all errors positive, and it penalises large errors far more than small ones (a prediction off by 10 contributes 100 to the loss; one off by 1 contributes only 1).

- When to use: Predicting continuous values — house prices, temperatures, stock prices.
- Caution: Very sensitive to outliers, since errors are squared. One extreme outlier can dominate the total loss.

Binary Cross-Entropy (BCE) — for Binary Classification

📖 Formula

$$\text{BCE} = -(1/n) \cdot \sum_i [y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i)]$$

$y_i \in \{0, 1\}$ (true binary label)

$\hat{y}_i \in (0, 1)$ (predicted probability from Sigmoid)

Measures how surprised the model is by the true label, given its prediction. High confidence in the correct answer gives near-zero loss. High confidence in the wrong answer gives loss approaching infinity.

BCE Intuition with Numbers

$y=1, \hat{y}=0.95$: term = $-\log(0.95) \approx 0.05$ correct and confident $\rightarrow$ tiny penalty

$y=1, \hat{y}=0.05$: term = $-\log(0.05) \approx 3.00$ wrong and very confident $\rightarrow$ large penalty

$y=0, \hat{y}=0.05$: term = $-\log(0.95) \approx 0.05$ correct and confident $\rightarrow$ tiny penalty

- When to use: Spam/not-spam, disease/healthy, fraud/legitimate — any 2-class classification.

Categorical Cross-Entropy (CCE) — for Multi-Class Classification

Formula

$$\text{CCE} = -(1/n) \cdot \sum_i \sum_k y_{ik} \cdot \log(\hat{y}_{ik})$$

K = number of classes

$y_{ik} = 1$ if sample i belongs to class k , else 0 (one-hot encoded label)

$\hat{y}_{ik}$ = predicted probability of class k from Softmax

A generalisation of Binary Cross-Entropy to $K > 2$ classes. Since $y_{ik} = 1$ only for the true class, the sum reduces to: $-\log(\text{probability assigned to the correct class})$. The network is penalised exactly for how low a probability it assigns to the right answer.

- When to use: Image classification (cat/dog/bird), digit recognition (0–9), language identification.
- Always pair with Softmax output activation — they are mathematically designed to work together.

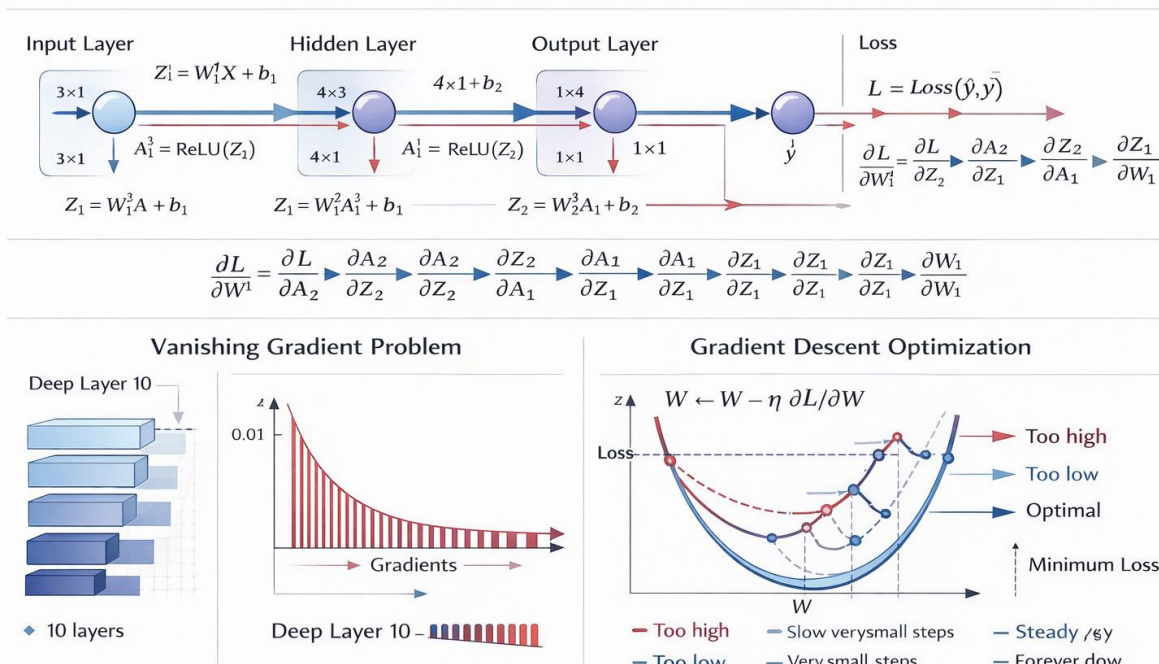
Loss Sanity Check at Initialisation

At random weight initialisation, Binary CE should be approximately $\ln(2) \approx 0.693$.

Categorical CE for K classes should be approximately $\ln(K)$ (e.g., $\ln(10) \approx 2.30$ for MNIST).

If your initial loss is much higher than these values, there is already a bug in your code or data pipeline.

Backpropagation & Gradient Descent



4.1 The Learning Goal

After the forward pass computes a prediction and the loss measures the error, the network must answer one critical question:

"Which weights contributed most to this error, and in which direction should each be changed to reduce the loss?"

This is precisely what backpropagation computes — the gradient of the loss with respect to every single weight in the network.

4.2 Backpropagation — The Chain Rule in Action

Backpropagation (Rumelhart, Hinton & Williams, 1986) applies the chain rule of calculus repeatedly, working backwards from the output layer toward the input layer, computing how much each weight contributed to the final loss.



Formula

Chain rule for a 2-layer network:

$$dL/dW^1 = (dL/dA^2) \cdot (dA^2/dZ^2) \cdot (dZ^2/dA^1) \cdot (dA^1/dZ^1) \cdot (dZ^1/dW^1)$$

Each term is a local gradient — easy to compute for each operation.

Multiplied together, they give the exact gradient of the loss w.r.t. any weight.

Step-by-step backpropagation:

1. Run the full forward pass and compute loss L .
2. Compute gradient of loss w.r.t. output activations: $\partial L / \partial \hat{y}$
3. Propagate gradient through output activation function: $\partial L / \partial Z^2$
4. Compute gradient w.r.t. output layer weights: $\partial L / \partial W^2 = \partial L / \partial Z^2 \cdot (A^1)^T$
5. Compute gradient w.r.t. output layer bias: $\partial L / \partial b^2 = \Sigma(\partial L / \partial Z^2)$
6. Propagate through hidden activation (ReLU): multiply by 1 if $z > 0$, else by 0.
7. Compute gradient w.r.t. hidden layer weights: $\partial L / \partial W^1$
8. Repeat until all layers are covered.

💡 Chain Rule Intuition

Think of the network as a chain of functions: Loss $\leftarrow$ Output activation $\leftarrow$ Output weights $\leftarrow$ Hidden activation $\leftarrow$ Hidden weights.

To find how a small change in W^1 affects the loss, you trace how that change ripples forward through every function in the chain.

The chain rule allows this efficiently by reusing local gradients — each is computed once and shared across all paths.

4.3 The Vanishing Gradient Problem

One of the most critical failure modes in deep networks, especially with Sigmoid or Tanh activations.

The problem in numbers:

- Sigmoid's maximum gradient is 0.25 (at $z = 0$). For $|z| > 2$, the gradient is near zero.
- Tanh's maximum gradient is 1.0 but drops sharply for $|z| > 2$.
- In a 10-layer Sigmoid network, the gradient reaching Layer 1 is approximately: $0.25^{10} \approx 0.000001$.
- Early layers receive almost no learning signal — they update at near-zero speed.

Consequences:

- Early layers fail to learn useful features.
- Training is unstable and extremely slow.
- Very deep networks with saturating activations are practically untrainable.

Solutions (detailed on Days 8 & 9):

- Use ReLU activations — gradient = 1 for positive z, largely eliminating saturation for active neurons.
- Batch Normalisation — keeps activations in a healthy range throughout training.
- Residual (Skip) Connections — provide gradient highways that bypass many layers.

4.4 Gradient Descent — Updating the Weights

Once gradients are computed, gradient descent updates every weight by taking a small step in the direction that reduces the loss. The negative sign is essential — we move opposite to the gradient (downhill, not uphill).

Formula

Weight update rule: $W \leftarrow W - \eta \cdot \partial L / \partial W$

W = current weight value

η (eta) = learning rate (a hyperparameter you set before training)

$\partial L / \partial W$ = gradient of the loss w.r.t. W (from backpropagation)

The Learning Rate η — The Most Critical Hyperparameter

Learning Rate	Behaviour	Outcome
Too high (e.g., 0.1–1.0)	Overshoots the loss minimum; loss oscillates wildly	Divergence — loss increases or never settles
Too low (e.g., 0.000001)	Tiny steps; very slow progress toward the minimum	Training takes too long; may not converge in time
Just right (e.g., 0.001)	Steady, consistent decrease in loss toward the minimum	Good convergence — smooth, decreasing loss curve

Gradient Descent Analogy

Imagine standing on a foggy hillside trying to reach the lowest valley.

The gradient tells you the steepest upward direction. You step in the opposite direction (steepest descent).

The learning rate is your step size. A giant step may overshoot the valley. A tiny step takes forever. A good step brings you steadily downhill.

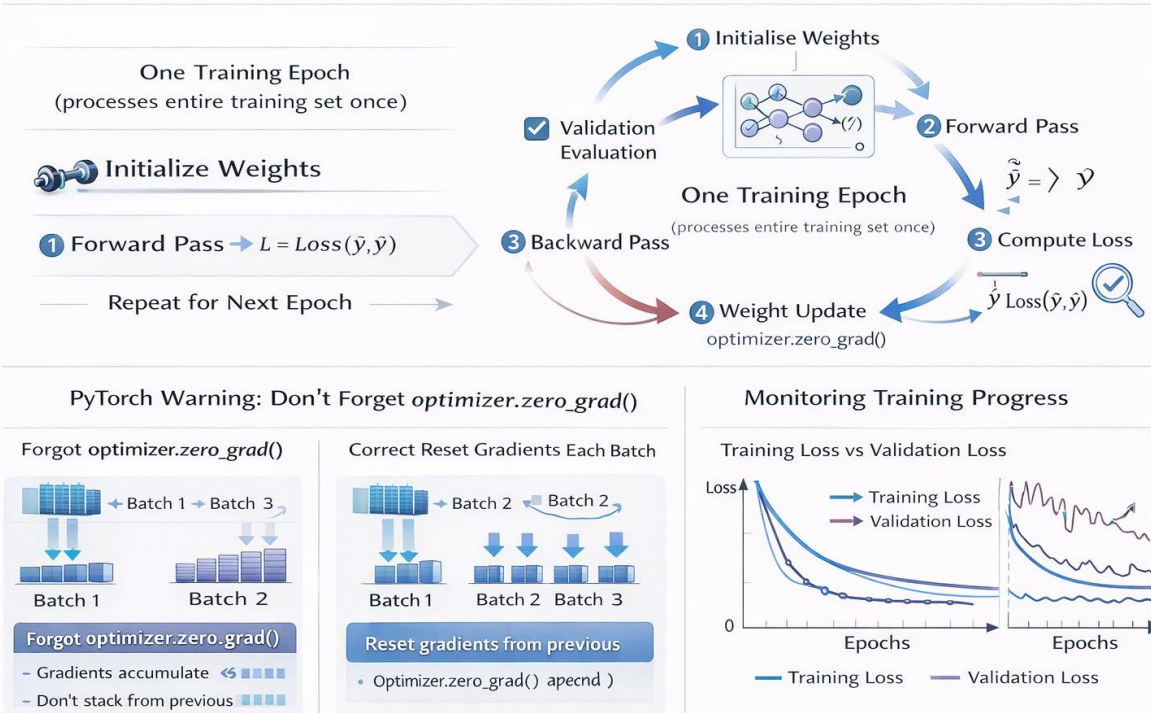
4.5 Variants of Gradient Descent

Variant	Samples Per Update	Pros	Cons
Batch Gradient Descent	Full dataset	Stable, accurate gradient	Very slow; memory-heavy for large datasets
Stochastic GD (SGD)	1 sample	Fast updates; noise can help escape local minima	Very noisy loss curve; slow overall convergence

Mini-Batch GD (standard)	32–256 samples	Balanced speed & stability; GPU-efficient	Requires tuning batch size
--------------------------	----------------	---	----------------------------

In modern deep learning, 'SGD' almost always refers to mini-batch gradient descent in practice. Batch size of 32 or 64 is a common starting point.

The Complete Training Loop



5.1 One Full Training Epoch

One epoch processes every sample in the training set exactly once. Here is the complete cycle, step by step:

Complete Training Loop — Pseudocode

1. Initialise all weights (e.g., He initialisation for ReLU networks)
2. For each mini-batch in the training set:
 - a. Forward pass → compute predictions $\hat{y}$ from input X
 - b. Compute loss → $L = \text{Loss}(\hat{y}, y)$
 - c. Backward pass → compute all gradients $\partial L / \partial W$ via backpropagation
 - d. Update weights → $W \leftarrow W - \eta \cdot \partial L / \partial W$
 - e. Zero gradients → reset all `.grad` values to 0 (critical in PyTorch!)
3. Evaluate on validation set → compute validation loss & accuracy
4. Repeat steps 2–3 for the next epoch until convergence or max epochs reached

PyTorch Zero-Grad Warning

In PyTorch, calling `loss.backward()` accumulates gradients into each parameter's `.grad` attribute.

If you forget `optimizer.zero_grad()` before each batch, gradients from previous batches pile up — causing incorrectly large weight updates.

Always call `optimizer.zero_grad()` at the very start of every batch loop iteration. This is the most common PyTorch bug for beginners.

5.2 Monitoring Training — What to Watch

Signal	What to Look For	What It Means
Training loss	Decreasing steadily each epoch	Network is learning — weights are updating correctly
Validation loss	Also decreasing, tracking training	Model is generalising well to unseen data
Gap: train vs val	Large and growing gap	Overfitting — model memorising training data (Day 8)
Loss not decreasing	Flat or oscillating loss	Check: learning rate, initialisation, data pipeline
Loss → NaN or Inf	Sudden explosion	Learning rate too high; add gradient clipping

06

Hands-On Lab — Build a 2-Layer Network in NumPy

Implement a 2-layer MLP (one hidden layer) from scratch using only NumPy — no PyTorch, no TensorFlow. Seeing every equation from today's lecture as actual working code deepens understanding far more than reading alone.

Lab Configuration

Parameter	Value
Dataset	XOR problem (4 samples) — then extend to MNIST subset
Architecture	Input (2 or 784) → Hidden (4 or 64, ReLU) → Output (1 or 10, Sigmoid/Softmax)
Loss	Binary Cross-Entropy (XOR) or Categorical Cross-Entropy (MNIST)
Optimiser	Mini-Batch Gradient Descent (hand-coded weight update)
Learning rate	0.01 (start) — experiment with 0.1 and 0.5 for XOR
Epochs	1,000 for XOR 50 for MNIST subset

Functions to Implement

NumPy Neural Network — Function Skeletons

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def relu(z):
    return np.maximum(0, z)

def relu_gradient(z):
    return (z > 0).astype(float) # 1 where z>0, 0 where z≤0

def forward_pass(X, W1, b1, W2, b2):
    Z1 = W1 @ X + b1
    A1 = relu(Z1)
    Z2 = W2 @ A1 + b2
    A2 = sigmoid(Z2)
    return Z1, A1, Z2, A2

def compute_loss(y_hat, y):
    # Binary CE: -mean(y*log(y_hat + 1e-8) + (1-y)*log(1-y_hat + 1e-8))
    return -np.mean(y*np.log(y_hat+1e-8) + (1-y)*np.log(1-y_hat+1e-8))

def backward_pass(X, y, Z1, A1, A2, W2):
    dA2 = A2 - y
    dW2 = dA2 @ A1.T
    db2 = np.sum(dA2, axis=1, keepdims=True)
    dA1 = W2.T @ dA2
```

```

dZ1 = dA1 * relu_gradient(Z1)
dW1 = dZ1 @ X.T
db1 = np.sum(dZ1, axis=1, keepdims=True)
return dW1, db1, dW2, db2

```

```

def update_weights(W, b, dW, db, lr):
    return W - lr * dW, b - lr * db

```

Expected Results & Debugging Guide

Symptom	Likely Cause	Fix
Loss not decreasing at all	Learning rate too low; all-zero weights	Try lr=0.1 or 0.5; check initialisation
Loss explodes to NaN/Inf	Learning rate too high; log(0) in loss	Reduce lr; add epsilon 1e-8 inside log()
Loss drops instantly to ~0	Data leakage or bug in loss function	Audit data split; verify loss formula
XOR loss stuck above 0.5	Network not deep enough or lr too high	Check architecture; try lr sweep
All predictions are the same	Weights initialised to zero (symmetry)	Use random He/Xavier initialisation

Extension challenges:

- Extend your network to 10-class MNIST. Add Softmax output layer and Categorical Cross-Entropy loss.
- Plot training loss vs. validation loss over epochs. Identify whether the model is underfitting, overfitting, or well-fitted.
- Swap ReLU with Sigmoid in the hidden layer. Observe and explain the change in training speed and final XOR accuracy.
- Implement mini-batch gradient descent. Compare training speed and loss curve stability against full-batch.



Key Terms Glossary

Term	Definition
Neuron (artificial)	The basic computational unit: computes a weighted sum of inputs, adds a bias, applies an activation function.
Weight	A learnable parameter that scales the influence of an input on a neuron's output.
Bias	A learnable scalar that shifts the activation threshold independently of the inputs.
Activation function	A non-linear function applied to the weighted sum to enable complex representations beyond linear models.
MLP	Multi-Layer Perceptron: a fully connected feedforward network with one or more hidden layers.
Forward pass	Computing a prediction by feeding data through each layer from input to output.
Loss function	A function measuring how far the network's prediction is from the true label; minimised during training.
Backpropagation	Algorithm for computing gradients of the loss w.r.t. all weights using the chain rule of calculus.
Gradient	Vector of partial derivatives of the loss w.r.t. each weight; points in the direction of steepest ascent.
Gradient Descent	Optimisation algorithm that updates weights opposite to the gradient to minimise the loss.
Learning rate (η)	Hyperparameter controlling the step size taken during each gradient descent update.
Epoch	One complete pass through the entire training dataset.
Mini-batch	A small subset of training data used to compute one weight update during mini-batch gradient descent.
Vanishing gradient	Gradients shrink exponentially in early layers of deep networks, preventing effective learning.
Dying ReLU	A neuron that outputs zero for all inputs and receives no gradient — permanently stuck at zero.



Review Questions & Self-Assessment

Conceptual Questions

1. Why can a single perceptron not solve the XOR problem? What architectural change is required, and why does it work?
2. What happens to a multi-layer network if all activation functions are removed and replaced with identity (linear) functions? Prove your answer algebraically.
3. Explain the vanishing gradient problem in detail. Which activation functions cause it and why? Which largely solves it, and how?
4. What is the role of the bias term b ? What would happen if every neuron in a network had no bias?
5. If your training loss is not decreasing after many epochs, list five possible causes and how you would diagnose and fix each.
6. Why do we subtract the gradient during the weight update step, rather than add it?

Calculation Questions

1. A perceptron has inputs $[0.5, -1, 2]$, weights $[0.3, 0.7, -0.2]$, and bias 0.1 . Compute the output using ReLU activation. Show all working.
2. For binary labels $y = [1, 0, 1]$ and predictions $\hat{y} = [0.9, 0.1, 0.4]$, compute the Binary Cross-Entropy loss manually.
3. A hidden layer has all weights initialised to zero. Explain in detail why this is catastrophic. What is the recommended initialisation for ReLU networks?
4. A 10-layer Sigmoid network: estimate the approximate gradient magnitude reaching Layer 1. Why would training fail?

Lab Extension Challenges

- Extend your NumPy network to classify MNIST digits (10 classes). Add Softmax output and Categorical Cross-Entropy loss.
- Plot training loss vs. validation loss over epochs. Identify whether your model is underfitting, overfitting, or well-fitted.
- Replace ReLU with Sigmoid in the hidden layer. Observe and explain the change in training speed and final XOR accuracy.
- Implement mini-batch gradient descent. Compare training stability and convergence speed against full-batch gradient descent.
- Add a second hidden layer to your NumPy network and observe whether it solves XOR faster or slower than a single hidden layer.